Experiment 9: CI/CD Pipeline with Testing & Deployment 🛠️

Local System Execution Guide (VS Code)

This guide walks you through setting up, running, testing, and demonstrating the complete **Experiment 9** full-stack project in **Visual Studio Code (VS Code)** on your local machine.

🌟 What Does This Project Demonstrate?

This project demonstrates an enterprise-grade **CI/CD Pipeline with Automated Testing & Staged Deployment**:

- **Backend (Spring Boot 3 + Java 17):**
  - **Assignment 1 — Unit Testing (JUnit 5):** Validates arithmetic and business logic independently (`CalculatorServiceTest`).
  - **Assignment 2 — API Integration Testing (MockMvc):** Simulates real HTTP requests (`GET /posts`, `POST /posts`) and asserts HTTP status codes (`PostControllerIntegrationTest`).
  - **Assignment 3 — Real Infrastructure Testing (Testcontainers):** Simulates ephemeral Docker containerized databases for production fidelity (`DatabaseContainerTest`).
  - **Assignment 4 — Containerization (Docker & Docker Compose):** Multi-stage `Dockerfile` and `docker-compose.yml` to package and run services portably.
  - **Assignment 5 — Canary Deployment:** Dynamic traffic routing engine between **Stable (v1.0.0)** and **Canary (v2.0.0)** with adjustable weight traffic splits (0% ➔ 10% ➔ 25% ➔ 50% ➔ 100%) and live telemetry.
  - **Contract Testing (Pact Concept):** Detects breaking API schema changes (e.g. backend sending `username` instead of `name`) before code reaches production.
  - **Automated Workflows:** GitHub Actions CI (`.github/workflows/ci.yml`) and CD (`.github/workflows/cd.yml`).
- **Frontend (React 18 + Vite):**
  - Interactive DevOps control center with a real-time traffic split slider, visual pipeline tracker, contract testing lab, batch traffic blast generator, and live post manager.

📋 Prerequisites

Before running the project in VS Code, ensure you have the following installed on your system:

1. **Java JDK 17 or higher** (e.g., OpenJDK / Temurin / Zulu 17, 21, or 23).

- Check in terminal: `java -version`

2. **Node.js (v18+) & npm**
   - Check in terminal: `node -v` and `npm -v`
3. **Maven** (Optional: a pre-configured `./mvnw` wrapper is already included in the `backend` folder).
4. **Docker Desktop** (Optional: only needed if you wish to run the app via Docker containers).

Project Directory Structure

```
Exp -9/
├── experiment_9.pdf        # Experiment syllabus and question paper
├── explanation.md          # In-depth theoretical explanation of all 10 topics
├── INSTRUCTIONS.md         # This step-by-step running guide
└── Code/
    ├── docker-compose.yml  # Docker Compose file for multi-service setup
    ├── .github/
    │   └── workflows/
    │       ├── ci.yml      # GitHub Actions CI automated build & test workflow
    │       └── cd.yml      # GitHub Actions CD containerize & canary rollout
    ├── backend/
    │   ├── Dockerfile      # Multi-stage production Dockerfile
    │   ├── pom.xml         # Maven configuration with Spring Boot & JUnit 5
    │   ├── mvnw / mvnw.cmd # Maven executable wrappers
    │   └── src/
    │       ├── main/       # Controllers, Services, Models, H2 Database config
    │       └── test/       # Unit, MockMvc, Contract, and Canary tests
    └── frontend/
        ├── package.json    # React & Vite configuration
        ├── vite.config.js  # Vite dev server configuration (port 5173)
        └── src/
            ├── App.jsx     # Interactive CI/CD & Canary UI Dashboard
            └── App.css     # Dark-mode responsive styling
```

Step-by-Step: How to Run in VS Code

We will run the project using **two split terminal windows** inside VS Code (one for the Spring Boot backend and one for the React frontend).

Step 1: Open the Project in VS Code

1. Launch **VS Code**.
2. Go to **File → Open Folder...**
3. Select the folder:

```
/Users/deepanshu/MyMac/CU_WORK/SEM-5th /Experiments/exp 5,6,7/Exp -9/Code
```

Step 2: Start the Spring Boot Backend (Terminal 1)

1. In VS Code, open a new integrated terminal by pressing `Ctrl + ~` (or `Cmd + ~` on macOS), or go to **Terminal → New Terminal**.
2. Navigate into the `backend` directory:

```
cd backend
```

3. Run the Spring Boot application using Maven:

```
./mvnw spring-boot:run
```

(Or if you have Maven installed globally, you can type `mvn spring-boot:run`).

4. **Success Check:** When you see the following log, your backend is live:

```
Tomcat started on port(s): 8080 (http) with context path ''
Started CicdDemoApplication in 2.1 seconds
```

5. You can verify it in your browser:
   - REST API Endpoints: http://localhost:8080/api/posts
   - Actuator Health Check: http://localhost:8080/actuator/health
   - In-Memory H2 Database Console: http://localhost:8080/h2-console
     - **JDBC URL:** `jdbc:h2:mem:cicddb`
     - **Username:** `sa`
     - **Password:** *(leave blank)*

Step 3: Start the React Frontend (Terminal 2)

1. In VS Code, split your terminal or open a second terminal tab by clicking the **+** or split icon in the top right of the terminal panel.
2. Navigate to the `frontend` directory:

```
cd frontend
```

3. Install dependencies (already prepared, but run this if you clone fresh):

```
npm install
```

4. Start the Vite development server:

```
npm run dev
```

database and reflected in the UI.

How to Run Automated Tests from VS Code Terminal

In a terminal window inside `Exp -9/Code/backend`, execute:

```
./mvnw test
```

(Or `mvn test`)

Test Output Summary:

All **18 automated tests** will execute and pass:

- **`CalculatorServiceTest` (6 tests):** Validates arithmetic and division-by-zero exceptions using JUnit 5 assertions.
- **`PostControllerIntegrationTest` (4 tests):** Tests HTTP endpoints `/posts`, `/api/v1/posts`, and `/api/v2/posts` with `MockMvc`.
- **`ApiContractTest` (2 tests):** Verifies compliant and breaking consumer-driven API contracts.
- **`CanaryDeploymentTest` (5 tests):** Validates default 10% weight, weight updates, 100% promotion, 0% rollback, and header overrides.
- **`DatabaseContainerTest` (1 test):** Simulates the Testcontainers Docker lifecycle.

```
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

🐳 Optional: Running via Docker (Assignment 4)

If you have Docker Desktop running:

Option A: Build and Run with Docker Compose

From the `Exp -9/Code` root directory:

```
docker-compose up --build
```

This will build the multi-stage Docker image and start:

- Main Service: `http://localhost:8080`
- Canary Staged Service: `http://localhost:8082`

Option B: Build Individual Docker Container

```
cd backend
docker build -t cicd-demo-app:v1.0.0 .
docker run -p 8080:8080 cicd-demo-app:v1.0.0
```

🔧 Troubleshooting & Tips

1. **Port 8080 already in use?**
    - Check what process is running on port 8080:

      ```
      lsof -i :8080
      ```

    - Kill the conflicting process or change `server.port=8081` in `src/main/resources/application.properties` and update `API_BASE` in `frontend/src/App.jsx`.
2. **Java Version Mismatch?**
    - Ensure `java -version` is JDK 17 or higher. The project targets Java 17+ bytecode.
3. **Frontend not connecting to backend?**
    - Ensure the backend is running first on port 8080 before clicking test buttons in the UI. The top-right badge will show **"Backend Online (Port 8080)"** in green.